

A STEP-BY-STEP TRAINING DECK

Python for Java Developers

From braces and classes to indentation and duck typing — one concept, one exercise, one model solution at a time.

12

Core Modules

36

Concept + Exercise + Solution
Slides

1

OOP Capstone

Prepared for evaluating & training advanced OOP developers moving to Python · Abinash Kumar Mishra

How to Run This Session

1

CONCEPT

Trainer walks through the idea and the Java-vs-Python comparison. Keep it short — 2–3 minutes.

2

EXERCISE

Trainer reads the task aloud, then steps back. Participants attempt it alone or in pairs while the exercise slide stays up.

3

PAUSE

Give real time — 5–10 minutes depending on module difficulty. Walk the room; don't rush to the solution.

4

SOLUTION

Reveal the model solution slide, walk through the key line, note the Java habit it replaces, then move on.

Modules covered

1. Syntax Mindset · 2. Variables & Dynamic Typing · 3. Collections · 4. Control Flow & Pythonic Loops · 5. Functions · 6. Classes & Objects
7. Instance vs. Class vs. Static Methods · 8. Inheritance & super() · 9. Encapsulation · 10. Polymorphism & Dunder Methods · 11. Abstract Classes & Interfaces · 12. Exceptions · 13.
Capstone: OOP System

1

MODULE 1 · CONCEPT

Syntax Mindset: Braces → Indentation

- Java uses { } to mark blocks and ; to end statements. Python uses indentation (whitespace) and a colon — no braces, no semicolons.
- A block is defined purely by consistent indent level; mixing tabs and spaces is a hard error, not a style nit.
- elif replaces “else if” — one word, no braces around the condition either.

JAVA vs PYTHON

JAVA

```
if (x > 0) {  
    System.out.println("positive");  
} else {  
    System.out.println("non-positive");  
}
```

PYTHON

```
if x > 0:  
    print("positive")  
else:  
    print("non-positive")
```

1

MODULE 1 · EXERCISE

Syntax Mindset: Braces → Indentation

Convert this Java grade classifier to Python. Keep the same logic: score $\geq$ 90 $\rightarrow$ "A", score $\geq$ 75 $\rightarrow$ "B", else $\rightarrow$ "C". Use proper indentation — no braces, no semicolons.

JAVA (CONVERT THIS)

```
int score = 82;
if (score >= 90) {
    System.out.println("A");
} else if (score >= 75) {
    System.out.println("B");
} else {
    System.out.println("C");
}
```

II PAUSE HERE — give participants time to attempt this before advancing

1

MODULE 1 · MODEL SOLUTION

Syntax Mindset: Braces → Indentation

```
score = 82
if score >= 90:
    print("A")
elif score >= 75:
    print("B")
else:
    print("C")
```

KEY POINT elif, not “else if”. Every block header ends in a colon.

► Discuss the key point, then move to the next module

2

MODULE 2 · CONCEPT

Variables & Dynamic Typing

- No type declaration — a name is a label bound to an object; the object carries the type, the variable doesn't.
- Reassigning a name to a different type is legal and common: `x = 5`, then `x = "five"` works fine.
- None is Python's null — a singleton object. Check it with ``is None``, not ``== None``.

JAVA vs PYTHON

JAVA

```
int x = 5;
String s = "hello";
x = 10;      // OK, same type
s = null;
```

PYTHON

```
x = 5
s = "hello"
x = "now a string"  # OK, rebinds
s = None
```

2

MODULE 2 · EXERCISE

Variables & Dynamic Typing

Write a function `describe(value)` that returns `type(value).__name__`. Call it with 5, 3.14, "hi", True, and None, printing each value alongside its type name.

STARTER

```
def describe(value):  
    ... # your code  
  
for v in [5, 3.14, "hi", True, None]:  
    ... # print v and describe(v)
```

II PAUSE HERE — give participants time to attempt this before advancing

2

MODULE 2 · MODEL SOLUTION

Variables & Dynamic Typing

```
def describe(value):  
    return type(value).__name__  
  
for v in [5, 3.14, "hi", True, None]:  
    print(v, "->", describe(v))
```

KEY POINT `type(v).__name__` mirrors Java's `v.getClass().getSimpleName()` — but works on any value, since even ints and bools are objects.

► Discuss the key point, then move to the next module

3

MODULE 3 · CONCEPT

Collections: List, Tuple, Dict, Set

- list $\approx$ ArrayList — mutable, ordered, resizable. Python has no separate array vs. ArrayList split.
- tuple is an immutable ordered sequence — no direct Java equivalent; closer to a fixed, hashable record.
- dict $\approx$ HashMap, set $\approx$ HashSet — both have first-class literal syntax: {}, {k: v}, {1, 2, 3}.

JAVA vs PYTHON

JAVA

```
List<Integer> nums = new ArrayList<>();
nums.add(1); nums.add(2);
Map<String,Integer> ages = new HashMap<>();
ages.put("Sam", 30);
```

PYTHON

```
nums = [1, 2]
ages = {"Sam": 30}
point = (3, 4)      # tuple
unique = {1, 2, 3}  # set
```

3

MODULE 3 · EXERCISE

Collections: List, Tuple, Dict, Set

Given the names list and scores dict below: add a new student "Zara": 88 to scores; build a list of names sorted by score descending; print the set of unique score values.

STARTER

```
names = ["Ravi", "Meera", "Ankit"]
scores = {"Ravi": 72, "Meera": 91, "Ankit": 65}

... # add Zara, sort names by score desc,
    # print the set of unique scores
```

II PAUSE HERE — give participants time to attempt this before advancing

Collections: List, Tuple, Dict, Set

```
names = ["Ravi", "Meera", "Ankit"]
scores = {"Ravi": 72, "Meera": 91, "Ankit": 65}

scores["Zara"] = 88
ranked = sorted(scores, key=scores.get, reverse=True)
unique_scores = set(scores.values())

print(ranked)
print(unique_scores)
```

KEY POINT `sorted(..., key=scores.get)` sorts dict KEYS using the VALUE as the sort key — a very common Python idiom with no direct Java one-liner.

► Discuss the key point, then move to the next module

4

MODULE 4 · CONCEPT

Control Flow & Pythonic Loops

- No classic C-style `for(int i=0;i<n;i++)` — Python's `for` is a for-each over any iterable. Use `range(n)` to count.
- `while` works exactly like Java's `while`.
- No do-while, and no classic switch — Python 3.10+ has match-case, but `if/elif` chains are still the everyday tool.

JAVA vs PYTHON

JAVA

```
for (int i = 0; i < 5; i++) {  
    System.out.println(i);  
}  
for (String s : list) {  
    System.out.println(s);  
}
```

PYTHON

```
for i in range(5):  
    print(i)  
  
for s in my_list:  
    print(s)
```

4

MODULE 4 · EXERCISE

Control Flow & Pythonic Loops

Print every number from 1 to 20 that is divisible by 3 or 5, using `range()` and the modulo operator — no manual index variable.

STARTER

```
... # loop from 1 to 20 inclusive
    # print n if divisible by 3 or 5
```

II PAUSE HERE — give participants time to attempt this before advancing

4

MODULE 4 · MODEL SOLUTION

Control Flow & Pythonic Loops

```
for n in range(1, 21):  
    if n % 3 == 0 or n % 5 == 0:  
        print(n)
```

KEY POINT `range(1, 21)` is start-inclusive, stop-exclusive — exactly like Java's `for (i=1; i<21; i++)`.

► Discuss the key point, then move to the next module

Functions: def, Defaults, *args/**kwargs

- def replaces the method signature boilerplate — no return type, no access modifier required.
- Default parameter values cover most of what Java uses method overloading for. Python has no overloading by signature.
- *args collects extra positional arguments into a tuple; **kwargs collects extra named arguments into a dict.

JAVA vs PYTHON

JAVA

```
// overloading required
int add(int a, int b) { return a+b; }
int add(int a, int b, int c) {
    return a+b+c;
}
```

PYTHON

```
def add(a, b, c=0):
    return a + b + c

def total(*args):
    return sum(args)
```

5

MODULE 5 · EXERCISE

Functions: def, Defaults, *args/**kwargs

Write `greet(name, greeting="Hello", **extra)` returning `f"{greeting}, {name}{punctuation}"`, where `punctuation` defaults to `!"` but can be overridden via `extra`. Call it three ways: default, custom greeting, and with punctuation override.

STARTER

```
def greet(name, greeting="Hello", **extra):  
    ... # your code  
  
print(greet("Ravi"))  
print(greet("Meera", greeting="Namaste"))  
print(greet("Ankit", punctuation="!!!"))
```

II PAUSE HERE — give participants time to attempt this before advancing

Functions: def, Defaults, *args/**kwargs

```
def greet(name, greeting="Hello", **extra):  
    punctuation = extra.get("punctuation", "!")  
    return f"{greeting}, {name}{punctuation}"  
  
print(greet("Ravi"))  
print(greet("Meera", greeting="Namaste"))  
print(greet("Ankit", punctuation="!!!"))
```

KEY POINT `extra.get("punctuation", "!")` is a one-line default lookup — no `getOrDefault()` ceremony needed.

► Discuss the key point, then move to the next module

Classes & Objects: `__init__` and `self`

- `class Car: ...` replaces Java's `class Car { ... }`. `__init__` is the constructor — always named `__init__`, never the class name.
- `self` is Python's explicit `'this'` — every instance method takes it as the first parameter, but you never pass it yourself.
- No `'new'` keyword — calling the class like a function creates and returns the instance.

JAVA vs PYTHON

JAVA

```
class Car {  
    String model;  
    Car(String model) { this.model = model; }  
    void honk() {  
        System.out.println(model+" says beep");  
    }  
}  
Car c = new Car("Civic");
```

PYTHON

```
class Car:  
    def __init__(self, model):  
        self.model = model  
    def honk(self):  
        print(f"{self.model} says beep")  
  
c = Car("Civic")
```

6

MODULE 6 · EXERCISE

Classes & Objects: `__init__` and `self`

Create class `BankAccount` with `__init__(self, owner, balance=0)`, a `deposit(self, amount)` method that adds to balance, and a `show(self)` method that prints "owner: balance". Create one account and deposit twice.

STARTER

```
class BankAccount:
    ... # your code

acc = BankAccount("Priya")
acc.deposit(500)
acc.deposit(250)
acc.show()
```

II PAUSE HERE — give participants time to attempt this before advancing

Classes & Objects: `__init__` and `self`

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

    def show(self):
        print(f"{self.owner}: {self.balance}")

acc = BankAccount("Priya")
acc.deposit(500)
acc.deposit(250)
acc.show()
```

KEY POINT `self.owner` / `self.balance` are created the moment you assign them in `__init__` — no field declarations above the constructor.

► Discuss the key point, then move to the next module

Instance vs. Class vs. Static Methods

- Instance methods take self and act on one object — the default, same idea as Java instance methods.
- @classmethod takes cls instead of self — operates on the class itself; commonly used as alternate constructors (Java's static factory methods).
- @staticmethod takes neither — a plain function namespaced inside the class, same as Java's static method.

JAVA vs PYTHON

JAVA

```
class Circle {  
    static int count = 0;  
    static double area(double r) {  
        return Math.PI * r * r;  
    }  
}
```

PYTHON

```
class Circle:  
    count = 0  
    @staticmethod  
    def area(r):  
        return 3.14159 * r * r  
    @classmethod  
    def from_diameter(cls, d):  
        return cls(d / 2)
```

7

MODULE 7 · EXERCISE

Instance vs. Class vs. Static Methods

Extend BankAccount (Module 6) with a class attribute `bank_name = "NovaBank"`, a `@classmethod` named `named(cls, owner)` returning a new zero-balance account, and a `@staticmethod` `is_valid_amount(amount)` returning `amount > 0`. Demonstrate all three.

STARTER

```
class BankAccount:
    bank_name = "NovaBank"
    ... # __init__ as before, plus
        # named() classmethod and
        # is_valid_amount() staticmethod

acc = BankAccount.named("Priya")
print(BankAccount.bank_name)
print(BankAccount.is_valid_amount(100))
```

II PAUSE HERE — give participants time to attempt this before advancing

Instance vs. Class vs. Static Methods

```
class BankAccount:
    bank_name = "NovaBank"

    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    @classmethod
    def named(cls, owner):
        return cls(owner, 0)

    @staticmethod
    def is_valid_amount(amount):
        return amount > 0

acc = BankAccount.named("Priya")
print(BankAccount.bank_name)
print(BankAccount.is_valid_amount(100))
```

KEY POINT `cls(owner, 0)` inside `named()` calls the constructor generically — it still works correctly if a subclass calls `named()` too.

► Discuss the key point, then move to the next module

Inheritance & super()

- `extends` becomes writing the parent class name in parentheses after the class name: `class Dog(Animal):`.
- `super().__init__(...)` calls the parent constructor — same idea as Java's `super(...)`, but always called explicitly by name.
- Python supports multiple inheritance directly; method resolution order (MRO) decides which parent's method wins on conflicts.

JAVA vs PYTHON

JAVA

```
class Animal {  
    void speak(){ System.out.println("..."); }  
}  
class Dog extends Animal {  
    void speak(){ System.out.println("Woof"); }  
}
```

PYTHON

```
class Animal:  
    def speak(self):  
        print("...")  
  
class Dog(Animal):  
    def speak(self):  
        print("Woof")
```

8

MODULE 8 · EXERCISE

Inheritance & super()

Create base class Shape with `__init__(self, name)` and `describe(self)` returning `f"This is a {self.name}"`. Create subclass `Square(Shape)` whose `__init__(self, side)` calls `super().__init__("square")` and stores side, plus an `area(self)` method. Print `describe()` and `area()`.

STARTER

```
class Shape:
    ... # your code

class Square(Shape):
    ... # your code, using super()

sq = Square(4)
print(sq.describe())
print(sq.area())
```

II PAUSE HERE — give participants time to attempt this before advancing

Inheritance & super()

```
class Shape:
    def __init__(self, name):
        self.name = name
    def describe(self):
        return f"This is a {self.name}"

class Square(Shape):
    def __init__(self, side):
        super().__init__("square")
        self.side = side
    def area(self):
        return self.side ** 2

sq = Square(4)
print(sq.describe())
print(sq.area())
```

KEY POINT `super().__init__("square")` must run before using `self.side` is optional, but running it first mirrors Java's rule that `super()` comes first.

► Discuss the key point, then move to the next module

Encapsulation: Convention, Not Keywords

- Python has no private/protected/public keywords. Convention signals intent: `_name` = internal use, `__name` = name-mangled (harder to hit by accident, not truly private).
- `@property` turns a method into an attribute-like accessor — replaces explicit `getX()/setX()` pairs while keeping validation logic.
- Everything is technically reachable; Python's philosophy trusts the programmer ("we're all consenting adults").

JAVA vs PYTHON

JAVA

```
class Person {  
    private int age;  
    public int getAge(){ return age; }  
    public void setAge(int a){  
        if (a >= 0) age = a;  
    }  
}
```

PYTHON

```
class Person:  
    def __init__(self, age):  
        self._age = age  
    @property  
    def age(self):  
        return self._age  
    @age.setter  
    def age(self, value):  
        if value >= 0:  
            self._age = value
```

9

MODULE 9 · EXERCISE

Encapsulation: Convention, Not Keywords

Create class `Temperature` with internal `_celsius`, a `@property` `celsius` getter, and a `celsius.setter` that raises `ValueError` if the value is below `-273.15` (absolute zero). Set a valid value, then catch an invalid one.

STARTER

```
class Temperature:
    ... # your code

t = Temperature()
t.celsius = 25
print(t.celsius)

... # try setting -300 and catch the error
```

II PAUSE HERE — give participants time to attempt this before advancing

Encapsulation: Convention, Not Keywords

```
class Temperature:
    def __init__(self, celsius=0):
        self._celsius = celsius

    @property
    def celsius(self):
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        if value < -273.15:
            raise ValueError("Below absolute zero")
        self._celsius = value

t = Temperature()
t.celsius = 25
print(t.celsius)

try:
    t.celsius = -300
except ValueError as e:
    print("Rejected:", e)
```

KEY POINT `t.celsius = 25` LOOKS like a plain field write, but it silently calls the `@celsius.setter` method — validation runs every time.

► Discuss the key point, then move to the next module

Polymorphism & Dunder Methods

- Polymorphism in Python leans on duck typing — “if it walks like a duck...”: no interface required, any object with the right method works.
- Java’s `toString()/equals()/hashCode()` map to Python’s `__str__`, `__eq__`, `__hash__` — “dunder” methods the interpreter calls automatically.
- `print(obj)` calls `__str__`; `obj1 == obj2` calls `__eq__` — override these instead of writing named methods.

JAVA vs PYTHON

JAVA

```
class Point {  
    public String toString(){  
        return "(" + x + "," + y + " ";  
    }  
    public boolean equals(Object o){ ... }  
}
```

PYTHON

```
class Point:  
    def __str__(self):  
        return f"({self.x},{self.y}) "  
    def __eq__(self, other):  
        return self.x == other.x \  
            and self.y == other.y
```

10

MODULE 10 · EXERCISE

Polymorphism & Dunder Methods

Create class `Point(x, y)` with `__init__`, `__str__` returning `"(x, y)"`, and `__eq__` comparing both coordinates. Create two Points with equal coordinates and one different; print them and compare with `==`.

STARTER

```
class Point:
    ... # your code

p1 = Point(1, 2)
p2 = Point(1, 2)
p3 = Point(3, 4)

print(p1)
print(p1 == p2)
print(p1 == p3)
```

II PAUSE HERE — give participants time to attempt this before advancing

Polymorphism & Dunder Methods

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __str__(self):
        return f"({self.x}, {self.y})"
    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

p1 = Point(1, 2)
p2 = Point(1, 2)
p3 = Point(3, 4)

print(p1)
print(p1 == p2)
print(p1 == p3)
```

KEY POINT Without `__eq__`, `==` would compare object identity (like Java's default `equals()`) and `p1 == p2` would print `False`.

► Discuss the key point, then move to the next module

Abstract Classes & Interfaces

- Java's interface / abstract class becomes Python's ABC (Abstract Base Class) from the abc module: inherit ABC, mark methods @abstractmethod.
- Python refuses to instantiate a class that still has unimplemented abstract methods (raises TypeError) — same spirit as Java's compiler check, enforced at runtime instead.
- Python has no separate 'interface' keyword — an ABC with only abstract methods plays that role.

JAVA vs PYTHON

JAVA

```
interface Payable {  
    double amount();  
}  
class Invoice implements Payable {  
    public double amount(){ return 100.0; }  
}
```

PYTHON

```
from abc import ABC, abstractmethod  
  
class Payable(ABC):  
    @abstractmethod  
    def amount(self): ...  
  
class Invoice(Payable):  
    def amount(self):  
        return 100.0
```

11

MODULE 11 · EXERCISE

Abstract Classes & Interfaces

Define abstract class `Notifier(ABC)` with abstract method `send(self, message)`. Create `EmailNotifier` and `SMSNotifier` subclasses, each implementing `send()` with a different printed format. Try instantiating `Notifier` directly and show it fails.

STARTER

```
from abc import ABC, abstractmethod

class Notifier(ABC):
    ... # your code

class EmailNotifier(Notifier):
    ... # your code

class SMSNotifier(Notifier):
    ... # your code

... # call send() on both, then try
    # Notifier() and catch the error
```

II PAUSE HERE — give participants time to attempt this before advancing

Abstract Classes & Interfaces

```
from abc import ABC, abstractmethod

class Notifier(ABC):
    @abstractmethod
    def send(self, message): ...

class EmailNotifier(Notifier):
    def send(self, message):
        print(f"EMAIL: {message}")

class SMSNotifier(Notifier):
    def send(self, message):
        print(f"SMS: {message}")

EmailNotifier().send("Server down")
SMSNotifier().send("Server down")

try:
    Notifier()
except TypeError as e:
    print("Cannot instantiate:", e)
```

KEY POINT The `TypeError` fires at instantiation time, not at class-definition time — Python checks abstractness only when you try to build an object.

► Discuss the key point, then move to the next module

Exceptions: try / except / else / finally

- Java's try/catch/finally becomes Python's try/except/finally, plus one addition: an else clause that runs only when NO exception occurred.
- Python catches by exception class, same idea as Java: `except IOError as e` reads like `catch (IOException e)`.
- Custom exceptions subclass Exception — same pattern as Java's custom Exception subclasses — but Python has no 'throws' declaration to satisfy.

JAVA vs PYTHON

JAVA

```
try {  
    risky();  
} catch (IOException e) {  
    handle(e);  
} finally {  
    cleanup();  
}
```

PYTHON

```
try:  
    risky()  
except IOError as e:  
    handle(e)  
else:  
    print("no error")  
finally:  
    cleanup()
```

12

MODULE 12 · EXERCISE

Exceptions: try / except / else / finally

Write `safe_divide(a, b)` returning `a/b`. Catch `ZeroDivisionError`, print a message and return `None`; use `else` to print "ok" when division succeeds; use `finally` to print "done" every time. Call it with `(10, 2)` and `(10, 0)`.

STARTER

```
def safe_divide(a, b):  
    ... # your code  
  
print(safe_divide(10, 2))  
print(safe_divide(10, 0))
```

II PAUSE HERE — give participants time to attempt this before advancing

Exceptions: try / except / else / finally

```
def safe_divide(a, b):  
    try:  
        result = a / b  
    except ZeroDivisionError:  
        print("Cannot divide by zero")  
        return None  
    else:  
        print("ok")  
        return result  
    finally:  
        print("done")  
  
print(safe_divide(10, 2))  
print(safe_divide(10, 0))
```

KEY POINT finally still runs even though the except branch returns None early — exactly like Java's finally running before a return actually leaves the method.

► Discuss the key point, then move to the next module

Build a Small OOP System

Combines every module above: abstract base class, __init__ validation, @property, abstract method, __str__, and inheritance across two subclasses.

Design an employee-management system: an abstract class Employee(ABC) with __init__(self, name, salary) that raises ValueError if salary < 0; a @property monthly_pay; an abstract method role(self); and __str__ returning "name - role: monthly_pay". Create subclasses Developer and Manager overriding role(). Build one of each and print them in a loop.

```
from abc import ABC, abstractmethod

class Employee(ABC):
    def __init__(self, name, salary):
        if salary < 0:
            raise ValueError("Salary cannot be negative")
        self.name = name
        self._salary = salary

    @property
    def monthly_pay(self):
        return self._salary / 12

    @abstractmethod
    def role(self):
        ...

    def __str__(self):
        return f"{self.name} - {self.role()}: {self.monthly_pay:.2f}"
```

```
class Developer(Employee):
    def role(self):
        return "Developer"

class Manager(Employee):
    def role(self):
        return "Manager"

team = [
    Developer("Ravi", 1200000),
    Manager("Meera", 1800000),
]
for person in team:
    print(person)
```

Mindset Recap

None of this is “Java with different punctuation.” Python trades compile-time guarantees for runtime flexibility — the discipline moves from the compiler to the developer.

Structure

Braces & semicolons → indentation & colons

Types

Declared types → dynamic typing, everything is an object

Access control

private/protected/public → naming convention (`_` and `__`)

Contracts

interface / abstract class → ABC + `@abstractmethod`

Object behavior

`toString()/equals()` → `__str__` / `__eq__` dunder methods

Overloading

Multiple signatures → default args & `*args/**kwargs`